
FLUENT AND COHESIVE: SUB-SECOND VOICE INTERACTION WITH GENERAL-PURPOSE OPEN-WEIGHT MODELS ON A 20-WATT EDGE DEVICE

A PREPRINT

Shaw Tan

Claire Tan

July 22, 2026

ABSTRACT

A voice assistant is fluent when its reply begins within the human turn-taking window — first audio in well under a second — and cohesive when the thing replying is a full large language model rather than an intent router. Production voice AI today gets one or the other: datacenter cascades median 1.4–1.7 s to first audio [1, 2], and the systems that beat the window do it with purpose-trained speech-to-speech models (Moshi [3]) that give up the text LLM’s generality. We show both are achievable at the same time, entirely on a 20 W, credit-card-sized edge board, using unmodified state-of-the-art open-weight models.

Our pipeline — streaming ASR (whisper.cpp), a Gemma 4 model (2B–12B) on a purpose-built ~6,000-line C/CUDA runner, and a VITS TTS (piper) — reaches 0.65 s from end of speech to first audio on a Jetson Orin NX 16GB responding to a 30-second spoken instruction, with the entire stack holding 4.2 GB of unreclaimable memory (it fits an 8 GB Jetson Nano) and peaking near 20 W. Four techniques carry the result, none of which modify the models: (1) prefill under speech — streaming ASR commits prefill into the open turn while the user is still speaking, backed by a proof that space-boundary splits are exact for SentencePiece, cutting post-speech TTFT from 5.37 s to 0.55 s on the 12B; (2) the LLM as clause splitter — a system prompt that makes the model create TTS flush points, motivated by the measured finding that the baseline model emits 21-word sentences with zero commas, leaving a clause-flushing TTS layer nothing to flush; (3) a streaming vocoder from stock TTS voices — ONNX graph surgery splits piper’s monolithic VITS at the latent boundary so the HiFi-GAN decoder (finite receptive field) runs in overlapped chunks, making first PCM O(1) in clause length (0.10 s vs 0.27–0.49 s), sample-exact against the monolithic decode; (4) multi-token-prediction speculative decoding characterized honestly as content-dependent (1.1–2.1×) and as a battery feature (2.29 vs 2.85 J/token). A cross-silicon study against Moshi on an RTX A5000 finds the remaining gap to speech-native models is structural (one decoded clause + one vocoder call, ~0.2 s) — and that the datacenter GPU buys only 1.7× over the 20 W board, because in the conversational regime the pipeline is floor-bound, not compute-bound.

Keywords voice assistants · edge computing · streaming ASR · speculative decoding · text-to-speech · large language models

1 Introduction

Conversation has a clock. Human turn transitions cluster tightly around a near-zero modal gap, with a cross-linguistic mean of roughly 200 ms [4]; beyond one second a reply reads as hesitation, and industry telemetry places the median production voice agent at 1.4–1.7 s with a P90 of 3.3–3.8 s (P95 4.3–5.4 s) — the region of talk-overs and user frustration [1, 2]. We name the two properties this paper pursues:

- **Fluent:** time from end of user speech to the first audio sample of the reply (TTFB) under one second — inside or adjacent to the human window.

- Cohesive: the reply is produced by a state-of-the-art general-purpose LLM in the loop — capable of instruction following, world knowledge, multi-turn context, tool use, and (in the same family) vision — not a latency-optimized intent model.

The two properties pull apart because of where each is cheap. Datacenter cascades are cohesive but pay network, queueing, and orchestration latency. Speech-native full-duplex models such as Moshi [3] are strikingly fluent — we measure ~ 130 ms TTFB on an RTX A5000 — but are purpose-trained speech models (a ~ 7 B Helium backbone fused to a neural codec), trained once, at great cost, with the conversational behavior baked in: no drop-in model upgrades, no text-domain tooling, no camera feed.

This paper takes the third corner: a cascaded pipeline of unmodified open-weight components — whisper.cpp ears, Gemma 4 brain, piper mouth — on a Jetson Orin NX 16GB, chosen as the robotics-relevant operating point (the AGX steps up in both power and cost; the Nano 8GB lacks the RAM headroom for the larger family members, though Section 5.6 shows our smallest configuration fits it). All Gemma 4 family models produce cohesive conversation out of the box; the entire challenge is speed, and the paper is the anatomy of where the time goes and how it is reclaimed.

Contributions:

1. Prefill under speech (Section 4.2): incremental prefill of an open turn fed by streaming-ASR commit semantics, with a correctness lemma — SentencePiece pieces never cross a space boundary, so word-boundary splits reproduce the one-shot tokenization exactly (verified: equal token ids, byte-identical replies). A 929-token spoken instruction costs 0.55 s TTFT after the last word instead of 5.37 s (12B; 0.10 s on the 2B).
2. The LLM as the clause splitter (Section 4.3): the split policy for streaming TTS must live in the model, because the measured baseline emits 21-word, comma-free sentences — there is nothing downstream to flush. A system prompt creates the flush points and cuts first audio from 1.21 s to 0.82 s; finetuning for human pause placement is the identified production path.
3. A streaming vocoder from stock voices (Section 4.4): the published piper ONNX is monolithic, but exactly one tensor crosses into its HiFi-GAN decoder; automatic graph surgery splits any stock voice there — no retraining — and the decoder’s finite receptive field makes overlapped-chunk decode sample-exact (max diff $4e-7$, two voices, two machines). First PCM becomes $O(1)$ in clause length: 0.10 s on the Orin where the monolithic call took 0.27–0.49 s. The full ablation ladder: first audio 1.21 $\rightarrow$ 0.82 (clause prompt) $\rightarrow$ 0.65 s (+streaming vocoder).
4. A latency-anatomy protocol for local voice agents (Section 4.1): the decomposition TTFT $\rightarrow$ time-to-first-speakable $\rightarrow$ TTFB, measured client-side under three delivery modes (deferred / burst / paced-realtime), with warmup-discard and byte-identity discipline. TTFT alone is shown to be uninformative for the spoken experience.
5. The whole-system result (Section 5): 0.65 s TTFB on 20 W; three latency/intelligence tiers from one runner (12B / E4B / E2B); 4.2 GB unreclaimable for ears+brain+mouth (8 GB-board feasible, with the measurement methodology that makes that number honest on Jetson); a decode rate that beats llama.cpp on the same board (1.17–1.26 $\times$) from $\sim 6,000$ lines of C/CUDA.
6. A cross-silicon study (Section 5.8) against Moshi on its own class of hardware, locating the cascade’s structural floor (one clause of decode + one VITS call) and showing the conversational regime is floor-bound: a datacenter GPU + desktop CPU improves end-to-end latency only 1.7 $\times$ over the 20 W board.

2 Related Work

Speech-native duplex models: Moshi [3] fuses a 7B text backbone with the Mimi streaming codec and full-duplex dialogue training; the paper itself reports a theoretical latency of 160 ms, 200 ms in practice [3] (the reference implementation attributes this practical figure to an NVIDIA L4 — 24 GB, ~ 72 W TDP datacenter card — a hardware detail from the project repository rather than the paper text [5, 6]); we measure ~ 130 ms TTFB warm on an RTX A5000 (trials 252/129/135 ms; the ~ 23 ms WebSocket handshake is excluded by construction — the clock starts when the client begins streaming audio, after the handshake byte). Two disclosures make this number conservative in Moshi’s favor: our run used the uncompiled PyTorch q8 path (torch.compile is broken on Windows), and the clock anchors at the start of input streaming, measuring the full-duplex loop latency ($\sim$ one 80 ms Mimi frame + one model step) rather than a turn response; Section 7 discusses the anchor mismatch with our end-of-speech metric. Mini-Omni and its successor [7, 8] follow the same species. These systems set the fluency bar, at the price of a fixed, purpose-trained model: the backbone is frozen into a speech topology, so the brain cannot be swapped for next quarter’s better open-weight release,

and text-ecosystem capabilities (tools, structured output, long instructions, vision) regress to what the speech training preserved.

Cascaded voice pipelines: Commercial stacks (Pipecat [9], LiveKit Agents [10]) cascade ASR $\rightarrow$ LLM $\rightarrow$ TTS and already stream at existing sentence punctuation. HuggingFace’s speech-to-speech (v0.2.10 [11]) is the closest published relative — open-weight, modular, LLM over an OpenAI-compatible HTTP endpoint — and we benchmark it head-to-head on our board with our exact LLM (Section 5.9). Our measurements locate the latencies these stacks leave on the table: prefill of the growing turn (hidden under speech, Section 4.2), the absence of early punctuation to flush on (created by the model itself, Section 4.3), and whole-sentence TTS synthesis before the first sample (Section 4.4).

Local inference runtimes: llama.cpp [12] is the gold-standard general local runtime, and its throughput numbers are the reference we benchmark against (Section 5.2). The pursuit of TTFB, however, is a different race than tokens/second: it rewards an appendable conversation abstraction (connection = conversation, line = turn, typed media/text frames into the open turn), speculative decoding tuned for short structured replies, and a server that streams the raw token channel for downstream clause flushing. little-gemma is not a llama.cpp competitor; it is a demonstration that a small, single-family runner can occupy a design point the general runtime does not: on the same Orin, matching $\sim 0.8\times$ of llama.cpp’s prefill while beating it $1.17\text{--}1.26\times$ in decode, and — the relevant number for this paper — reaching first-sentence $18\text{--}48\times$ earlier on multimodal turns (Section 5.2, thought suppression and encode-off-critical-path).

Streaming ASR: whisper_streaming / LocalAgreement-2 [13] provides the commit semantics our prefill rides on; the ASR side is prior art we consume, not a contribution.

Speculative decoding: EAGLE [14], Medusa [15], self-speculation. Gemma 4 ships a first-party MTP head; our contribution is not the mechanism but the honest characterization (content- and head-width-dependence) and the energy angle (Section 4.5), which the edge literature largely lacks.

3 System

Hardware: Jetson Orin NX 16GB module (Ampere iGPU, shared LPDDR5), NVMe SSD, JetPack 6.2.1, headless. Rationale: the NX is the robotics operating point — credit-card footprint, $10\text{--}25\text{ W}$ envelope (40 W MAXN available), $\sim \$600$ -class as a bare module (1KU pricing) or $\sim \$900$ -class as a complete carrier-board dev kit [16]. The AGX buys performance with a power/cost step that changes the product class; the Nano 8GB cannot hold the 12B tier (Section 5.6 qualifies what it *can* hold).

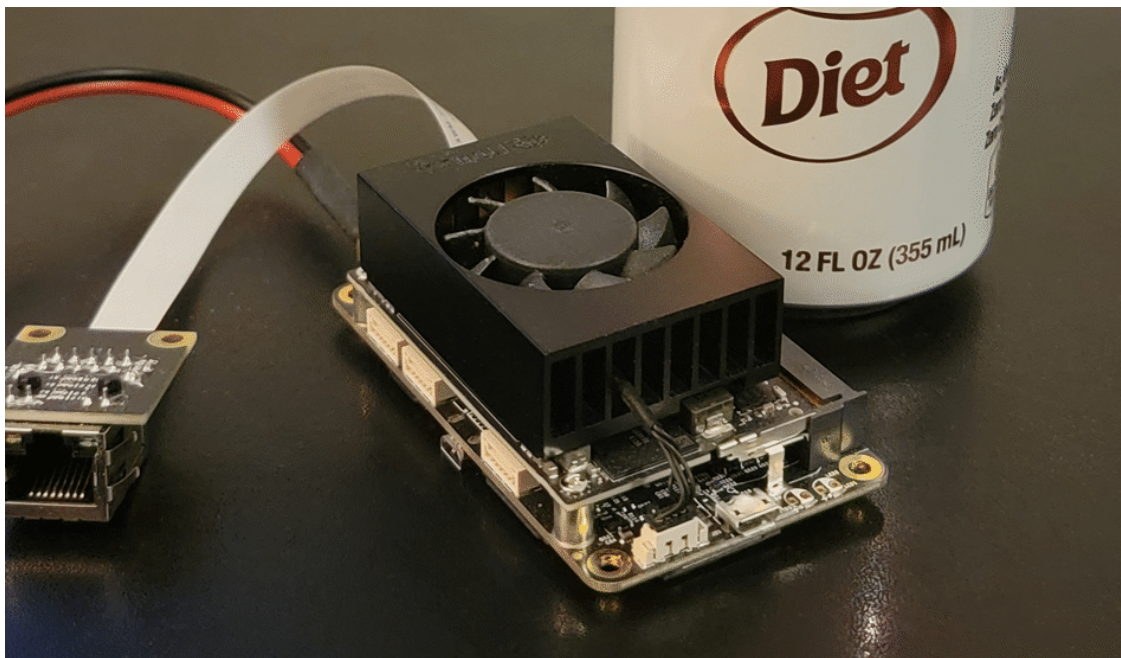


Figure 1: The system under test. Every edge number in Section 5 was measured on this board — a Jetson Orin NX 16GB on its carrier board (fan attached), 355 mL can for scale. The compute module itself is $69.6 \times 45\text{ mm}$, smaller than a credit card; this is the entire machine that answers 0.65 s after end of speech.

Three processes, one GPU owner:

- voicecat + whisper.cpp (CUDA base.en): energy VAD, streaming transcription with LocalAgreement-2 commits, timestamp-based window trimming, barge-in (one byte stops the reply mid-stream). Committed words stream into the runner as typed text frames while the turn is still open.
- Gemma 4 on little-gemma: a $\sim 6,000$ -line C/CUDA runner for the Gemma 4 family (int8 tensor-core prefill, dp4a decode, GPU vision encoder, MTP speculative decoding, all quantizations byte-identical to the f32 reference by gate). The serve mode is a Unix-domain socket: connection = conversation, line = turn, typed frames ('T' text, 'I' image, 'A' audio) append to the open turn and prefill on arrival. A system-prompt file prefills once at server start (Section 4.3 rides this).
- piper (VITS, en_US voices) as a persistent stdin service on the CPU — the GPU belongs to the LLM. One ONNX call per line: first byte is roughly equal to whole-clause synthesis, 0.27–0.49 s per clause on the Orin's cores (18–26 $\times$ realtime), 0.31 GB peak anonymous memory. Piper was chosen over Kokoro on speed (0.008 RTF vs ~ 0.47 –0.51 RTF) [17]. CPU placement is measured, not assumed: on the A5000, CUDA piper loses to a desktop CPU at clause length (0.112 vs 0.089 s — the ONNX graph round-trips the bus 28 times for ops pinned to CPU, and launch/copy overhead swamps 20 MB of conv math) and only reaches 1.6 $\times$ on long sentences; clause splitting (Section 4.3) moves TTS precisely into the regime where CPU wins, and on the Orin the iGPU would contend with decode for the same LPDDR5 anyway. Section 4.4 removes the whole-clause wait itself: the same voice files, split at the latent boundary, stream first PCM in 0.10 s.

Why external ASR when Gemma 4 is natively multimodal? Because when we built it, the native path failed three ways (Section 7 for details): the E2B/E4B conformer audio path confabulated rather than transcribed; the 12B hears speech but behaves as ASR-only for non-speech sound; and — the finding we could not have predicted — vision in context disables hearing on the 12B (seven arrangements tested, asymmetric, session-scoped, survives black frames). Mid-project, Google's QAT releases repaired the first failure: the fault was the original mmproj's audio-encoder export, and the QAT-era encoder transcribes verbatim once its exported fake-quant activation ranges are applied (Section 5.5) — so the same engine now also runs whisper-free on the E2B and E4B tiers, streaming the model's own ears to 0.46 s first audio. The other two failures stand, and they scope the two lanes: whisper remains the lane wherever a session may see frames or needs non-speech awareness (music, ambient — audio rides as a whisper transcript under the video span), and for the 12B tier; native ears are the speech-only, vision-free option that removes the ASR process entirely. The flexibility itself is the point: the ears, like the brain, are a swappable part.

4 Where the time goes, and how it is reclaimed

4.1 Metrics and protocol

- TTFT: last user input $\rightarrow$ first reply token (server work becomes visible).
- TTFS / time-to-first-speakable: $\rightarrow$ first complete speakable unit (sentence, or clause once Section 4.3 applies) — the unit the TTS can start on.
- TTFB (first audio): $\rightarrow$ first PCM sample out of the TTS. The number the ear judges. TTFT alone misorders systems: a fast first token followed by a 21-word sentence is slower to speak than a slower first token followed by a 6-word clause.

All clocks are client-side (the runner is not trusted to grade itself), the first turn after model load is discarded (repack, clock ramp, draft-head warmup), and each measurement is a fresh conversation. Three delivery modes model the ASR: deferred (whole instruction arrives at turn end — the no-streaming baseline), burst (frames arrive as fast as the socket takes them), paced (30 words/s with 200 ms frame cadence — a real speaking rate). The 929-token spoken instruction (a long rambling preamble ending in a question) is the standard stimulus; replies must be byte-identical across delivery modes for a run to count (Section 4.2's lemma makes this achievable and it held in every reported cell, on both test machines).

4.2 Prefill under speech

Chunked prefill — a named scheduler feature in vLLM and TensorRT-LLM — splits a completed prompt so it can interleave with decode; llama.cpp's server has no such feature but likewise processes a prompt that has already arrived in full (ubatch splitting, prefix-cache reuse). In every serving stack we know, prefill begins only once the whole prompt is in hand. The open turn is different: while the user speaks, committed words are already immutable —

LocalAgreement-2 commits exactly the prefix both hypotheses agree on — so their KV entries can be computed now, under the speech itself.

Correctness requires that tokenizing the turn in committed pieces equals tokenizing it whole. For SentencePiece this holds exactly when splits land on space boundaries: pieces never cross a space (the space glyph `_` starts pieces), so a word-boundary split cannot merge or reshape tokens across the seam. We verify, not just argue: equal token id sequences and byte-identical replies, deferred vs streamed, all three models. (One span of holdback is kept unprefilled to absorb ASR revisions of the most recent words; MSG_PEEK lets the server prefill right up to the pending frame without consuming it.)

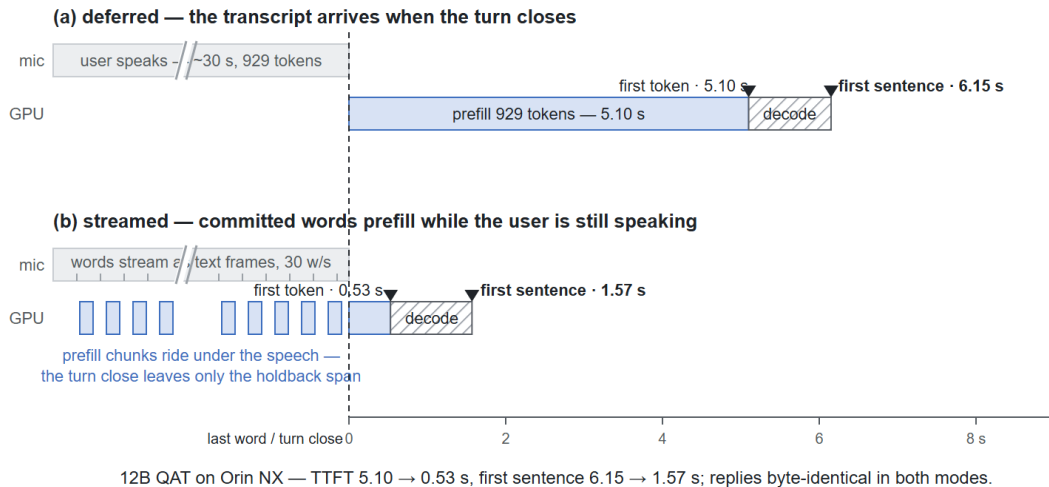


Figure 2: In deferred delivery (a) the 5.37 s prefill starts only when the turn closes; streamed (b), committed words prefill during the speech itself and the turn close leaves only the holdback span — TTFT 5.37 → 0.55 s with byte-identical replies.

Measured on the Orin (TTFT after last word, deferred → paced): 12B 5.10 → 0.53 s, E4B 2.04 → 0.16 s, E2B 1.15 → 0.10 s. The mechanism generalizes beyond dictation: camera and video frames prefill on arrival the same way (media spans, 3.0× video TTFT), which is what makes the same pipeline camera-capable — and, once the QAT releases repaired the family’s native audio encoder, raw audio chunks ride it too (Section 5.5, TTFT flat in utterance length with zero engine changes) — the flexibility argument against purpose-trained speech models made concrete.

A protocol note: at a real speaking pace the hiding is essentially free — prefill chunks slot into the pauses between frames. Burst delivery pays padding costs and is the wrong choice when the full text is already in hand; send it as one line instead (measured, Section 5.3 table note).

4.3 The LLM is the only clause splitter

A streaming TTS layer flushes at punctuation. The baseline 2B, asked our standard question, answers in a single 21-word sentence with zero commas — there is nothing to flush until the sentence ends, 9.4 s of audio in one synthesis call. No glue-layer heuristic fixes this: splitting mid-clause produces unnatural speech, and only the model knows where a thought can pause. The split policy therefore lives in the model, installed as a system prompt loaded once at server start (prefilled once, zero per-turn cost):

You are a voice assistant. Everything you write is spoken aloud by a text-to-speech engine, clause by clause, so write for the ear, not the eye. [...] Keep sentences short. Break long thoughts into clauses of five to ten words, separated by commas or periods [...] Get to the point in the first clause; details can follow.

Effect on the E2B voice loop: first speakable unit 21 → 8 words (0.714 → 0.549 s), synthesis of that unit 0.49 → 0.27 s (VITS synthesis is ~linear in audio length, so shorter first units pay twice), first audio 1.21 → 0.82 s. Two honest notes travel with this: style pressure sharpens a small model into confident error (the 8-word answer contains a wrong

date; the vague baseline was defensible), and prompt-only is the bootstrap — the production path is a light finetune for human pause placement, which is often shorter than grammatical clauses.

4.4 Streaming vocoder from stock voices

After Section 4.2 and 4.3, the TTS call itself is 30–50% of what remains: VITS synthesizes a whole clause in one ONNX call, so first PCM waits for all of it. The fix exploits an asymmetry inside the model. The text encoder, duration predictor, and flow must see the whole clause — prosody is global — but they are cheap; the HiFi-GAN upsampler is most of the compute and is a pure convnet: finite receptive field, no attention, no recurrence. Anything with a finite receptive field can be decoded in overlapped chunks whose interiors are exact.

piper ships monolithic ONNX files, but the graph names its decoder, and exactly one tensor crosses into it (the masked latent feeding the decoder’s input conv). Our splitter finds that boundary automatically and extracts encoder/decoder halves from any stock voice file — no retraining, no re-export; verified identical on an official voice and on our own finetune. At overlap $R=16$ frames the chunked decode saturates at max sample difference $4e-7$ vs the monolithic output (fp32 noise), on both test machines. The streaming service (a drop-in for piper –output-raw) then emits 0.12 s chunks as they decode. The implementation ships in our piper fork as `python -m piper.split` plus a `–stream` CLI flag and `synthesize_stream()` API, with an exactness test suite; it is a candidate for an upstream contribution.

End-to-end on the Orin, everything included (phonemization, encoder, first chunk, pipe): first PCM 0.10 s where the monolithic call took 0.27 s (voice-sys clause) — and 0.15 s even for a 21-word sentence whose monolithic call took 0.49 s. First byte is now $O(1)$ in clause length; synthesis continues at 6–13× realtime under playback. The techniques compose: first audio $1.21 \rightarrow 0.82$ (Section 4.3) $\rightarrow 0.65$ s. One trade is inherent: streamed chunks cannot be peak-normalized (that needs the whole clause), so the service ships the voice’s native level. This is the one architectural idea the cascade borrows from speech-native models: their codecs stream by construction; ours now does too.

4.5 Speculative decoding as a latency and battery feature

Gemma 4’s first-party MTP head drafts the next N tokens; greedy verification makes output byte-identical to plain decoding by construction — only tokens/s and acceptance move. What it is worth is content-dependent (block-3, serve mode, steady-state, QAT builds): structured output 2.0–2.3× (Orin E4B 20.7 $\rightarrow$ 48.6 tok/s at 99% acceptance), code 1.9×, image description 1.5× (measured on image-only turns — speculation survives vision context intact), free prose 1.40–1.44× (the 12B’s 1024-wide head lands ~51% of prose drafts; E4B’s 256-wide head ~37% — MTP helps chat more as the model grows). We report the distribution, not a flat multiplier — flat claims in the spec-decoding literature do not survive contact with prose.

Getting prose past $\sim 1.1\times$ took a kernel observation: the verify batch ($B = 2-5$) is its own regime — too narrow for tensor-core prefill tiles, priced per-column on the decode matvec — and a verify kernel that shares each column’s activation registers across two output rows drops $B=2$ verify to $\sim 1.01\times$ the cost of a decode step (details in the repository journals [18]). With it, the same board and models out-generate llama-server’s own draft-mtp speculative path at every content type measured, same day, same GGUFs (prose 29.9 vs 24.5 tok/s; code 40.7 vs 34.1; image description 31.5 vs 28.8). On the battery angle: MTP cuts energy per token from 2.85 to 2.29 J/token on the Orin (the GPU is bandwidth-bound; fewer full passes per token is less DRAM traffic; measured pre-kernel — the cheaper verify only widens it) — for a robot, spec decoding is a battery feature before it is a latency feature.

4.6 The substrate, briefly

None of Section 4.2–4.5 works if the kernels are slow. The runner’s prefill is an int8 tensor-core (mma) path with warp-cooperative activation quantization and flash-attention prefill; decode is dp4a with split-K attention, frozen kernels, and CUDA-graph capture; weights are zero-copy on the integrated GPU (host and device share LPDDR5 — copying would hold every byte twice), mmap’d and unpinned where the quantization is repacked anyway. Every optimization is gated byte-identical against a f32 reference and, for the speculative path, by acceptance-rate tripwires (which caught three real bugs). Full engineering logs, including the falsified dead ends, are in the repository journals [18]. The result on the Orin: 0.82–0.86× of llama.cpp’s prefill throughput and 1.08–1.11× its decode on the QAT defaults (1.17–1.27× on the earlier Q4_K_M builds), with MTP-on generation ahead of llama-server’s own speculative path — from a codebase a single reviewer can actually read ($\sim 6,000$ lines).

5 Evaluation

5.1 Setup

Orin NX 16GB, JetPack 6.2.1, pinned clocks for benchmarks (jetson_clocks; production ships DVFS — a reboot resets the pin, and every number here was re-verified against that trap), models: Gemma 4 QAT q4_0 releases across the family — 12B-it, E4B-it, E2B-it (Google’s quantization-aware checkpoints, unsloth GGUFs), each with its matched MTP head; whisper.cpp CUDA base.en; piper en_US voices. Cross-silicon: RTX A5000 (Windows) + i7-8086K. All measurements are client-side per Section 4.1. The E4B and 12B rows in Sections 5.2, 5.3 and 5.5 were re-measured 2026-07-19 on the QAT builds (which sped up both stacks — llama.cpp’s q4_0 decode gains more than ours); the multimodal comparison in Section 5.2 and the cross-silicon study (Section 5.8) predate the switch (Q4_K_M builds) and are labeled — the switch only improves our side of those.

5.2 The substrate vs llama.cpp

device	model	little-gemma	llama.cpp	ratio
Orin	12B	193	232	0.84×
Orin	E4B	474	553	0.86×
Orin	E2B QAT	834	1,020	0.82×

Table 1: Serve-mode prefill, tok/s (929-token turns, warm, first discarded; llama.cpp = llama-bench pp929, best of fa 0/1, same-day pairs))

device	model	little-gemma	llama.cpp	ratio
Orin	E4B QAT	20.7	18.7	1.11×
Orin	12B QAT	9.8	9.05	1.08×
Orin	E2B QAT	34.5	37.4	0.92×

Table 2: Plain decode, tok/s (serve mode, 1,024-token replies — sustained over 0–1K context; llama-bench tg128 at matched depth, best of fa 0/1))

(Two honest notes. The serve numbers include our per-token serving overhead while llama-bench measures bare kernels, so the ratios are conservative. And q4_0 is llama.cpp’s oldest, most heavily tuned decode quant — on the earlier Q4_K_M builds our lead was 1.17–1.27×

— so the QAT switch, which speeds up both stacks in absolute terms, narrows the ratio; we take the faster absolute tokens.) MTP multiplies on top, and there the comparison inverts decisively: with each family’s first-party MTP head engaged, little-gemma out-generates llama-server’s own speculative path at every content type measured (Section 4.5). Where the design point shows most is multimodal turns: little-gemma reaches first-sentence on an image+question turn in 1.98 s (E4B) / 5.22 s (12B) on the Orin vs 35.8 / 98.6 s for llama-server (which burns its lead in an unsuppressed thinking channel and host-side encode; Q4_K_M-era measurement — the QAT builds only shorten our side) — the raw-token-stream server and encode-off-critical-path choices are TTTFB choices, not throughput choices.

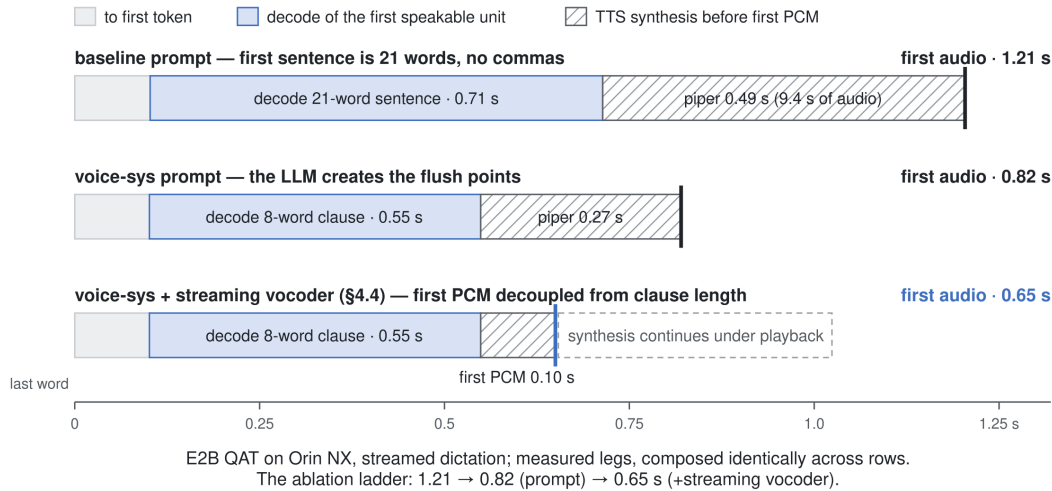
5.3 The dictation matrix (Orin)

model	deferred	paced (30 w/s)
12B	5.10 / 6.15	0.53 / 1.57
E4B	2.04 / 2.93	0.16 / 1.05
E2B QAT	1.15 / 1.76	0.10 / 0.72

Table 3: 929-token spoken instruction; TTFT / TTFS in seconds

(Burst delivery pays frame padding and is dominated by deferred — measured, not assumed; deferred runs reproduce to the millisecond.)

5.4 End-to-end: the 0.65 s anatomy



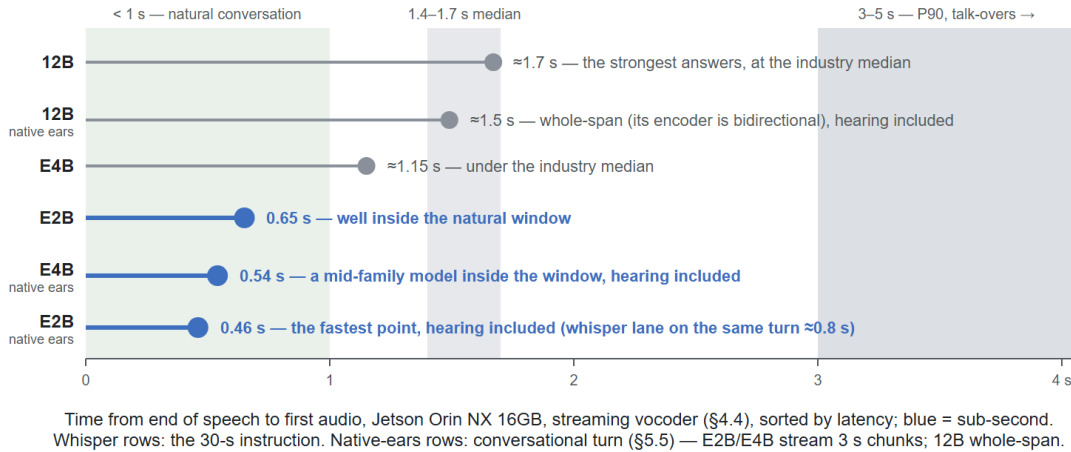


Figure 4: One board, one pipeline, three operating points. The E2B tier lands inside the natural-conversation window that production telemetry reports as rarely achieved; the E4B matches the industry median; the 12B trades latency for the strongest answers.

Native hearing. The QAT checkpoints also restore the E2B and E4B native audio path. The original failure traced to the exported `mmproj` audio encoder: the pre-QAT checkpoints entered token loops because exported fake-quant activation ranges were omitted. Applying the QAT-era exported clamp values (120 per-projection activation scales) restores verbatim transcription, even when paired with the earlier language backbone.

Our implementation follows the same streaming architecture described in Section 4.2: log-mel features are computed on the host while the 12-block causal conformer executes on the tensor cores. Warm-up is approximately 0.2 s for an 11 s recording, and arriving audio chunks prefill exactly like text, requiring no inference-engine changes.

The conformer determines the chunking behavior. Empirically, 3 s chunks preserve verbatim transcription across chunk boundaries, whereas 2 s chunks occasionally omit words and 1 s chunks collapse entirely. With 3 s chunking, TTFT remains essentially independent of utterance length—the same signature observed for text streaming in Section ?? . Table 5 summarizes the measured end-to-end results.

Model	Delivery	Utterance	TTFT	TTFS	First audio
E4B	Deferred (one frame)	4.5 s	0.43	0.75	≈0.85 s
E4B	Streamed (3 s chunks)	4.5 s	0.24	0.44	≈0.54 s
E4B	Streamed (3 s chunks)	11 s	0.25	0.64	≈0.74 s
E2B	Streamed (3 s chunks)	4.5 s	0.15	0.36	≈ 0.46 s
E2B	Streamed (3 s chunks)	11 s	0.16	0.53	≈0.63 s
12B	Deferred (whole span)	4.5 s	0.78	1.39	≈1.49 s

Table 5: Native-hearing latency, measured on warm runs with client-side clocks. The E2B reaches first audio in ≈0.46 s including speech recognition, the fastest configuration evaluated.

Including hearing, the E2B reaches first audio in ≈ 0.46 s, the fastest operating point in the paper. The E4B follows at ≈ 0.54 s, while the 12B answers in ≈ 1.5 s despite using its unified audio path. Native hearing therefore shifts every model tier downward by roughly one latency class while eliminating a separate speech-recognition process.

The architectures differ in how they stream. The E2B and E4B use a causal conformer (5-tap causal convolutions with a 12-frame causal attention window), allowing incremental chunked inference. The 12B instead employs a bidirectional unified encoder over the full audio span; forcing chunked operation causes the model to enter repetitive reasoning loops, so inference is intentionally deferred until the complete utterance arrives.

Two evaluation details accompany these measurements. First, the E2B achieves verbatim transcription through the streamed 3 s chunking path, whereas a single uninterrupted 11 s span truncates, making chunking beneficial even aside from latency. Second, the 12B’s verbatim transcription was verified only on the whole-span path.

Whisper nevertheless remains the default deployment. Beyond robustness to non-speech audio, a transcript is editable text between the acoustic front-end and the language model: applications can inspect, filter, annotate, or correct the transcription before it is consumed. Native hearing instead minimizes latency by removing that intermediate representation. At present it is therefore offered as the speech-only operating mode. Current limitations are checkpoint-specific rather than architectural: non-speech audio remains unreliable, simultaneous vision and hearing still interfere, and the chunk-size study has so far been validated on a single reference recording.

5.6 Memory: does it fit an 8 GB Nano?

Measured the honest way for Jetson (anonymous RSS + nvmap; VmHWM counts evictable file pages and MemAvailable under-counts nvmap — both mislead): little-gemma E2B QAT serve 3.2 GB unreclaimable (nvmap 3.00 + anon 0.11, after mmap'ing the weight blob and skipping the pin for fully-repacked models: 5.5 → 3.2 GB), whisper ~0.7 GB, piper 0.31 GB peak → ≈ 4.2 GB for the full voice stack, leaving ~2.2 GB slack on a 7.4 GB-usable headless Nano, with room for the vision encoder (~0.7 GB). Caveat, stated plainly: measured on the NX 16GB against an 8 GB budget, not yet on the Nano SKU.

5.7 Power

GPU-saturated decode peaks ~23 W (default profile; 40 W MAXN raises clocks but the workload is bandwidth-bound), idle-to-peak on a supply a robot already carries. MTP reduces J/token 2.85 → 2.29 (Section 4.5). Moshi's reference deployment is an L4 (24 GB, ~72 W TDP) datacenter GPU.

5.8 Cross-silicon: the cascade vs Moshi on Moshi's hardware class

model	1st token	1st clause	+piper	word → audio
12B	0.090	0.241	0.109	0.35 s
E4B	0.065	0.132	0.108	0.24 s
E2B QAT	0.121	0.29	0.193	0.48 s

Table 6: Moshi on our A5000: ~0.13 s TTFB warm. Our pipeline on the same card (+ i7-8086K for piper), same protocol as Section 5.2

Three readings. (1) The remaining 2–4× is structural: a speech-native model starts vocoding without waiting for a speakable text unit; the cascade's floor is one decoded clause plus one VITS call (~0.2 s). (2) Model size has almost stopped mattering — 2B to 12B spans first-clause 0.13–0.29 s once streaming hides prefill; content dominates (the E2B is slowest end-to-end because its clause contains a date that verbalizes to 3.9 s of audio). (3) The datacenter card + desktop CPU beat the 20 W board by only ~1.7× end-to-end (0.48 vs 0.82 s, monolithic-TTS configurations — the streaming vocoder shipped after this study and tightens both sides): the conversational regime is floor-bound, not compute-bound, which is precisely the regime where an edge deployment loses nothing that matters.

5.9 Same board, same brain: the cascade baseline

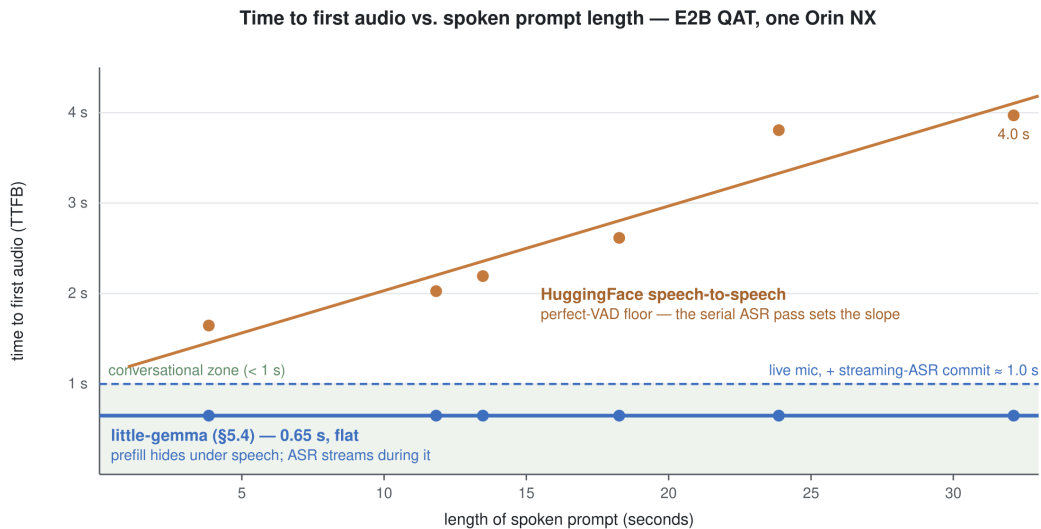
The cleanest ablation of our techniques is another cascade with everything else held equal. HuggingFace's speech-to-speech pipeline ran on the same Orin NX against the same Gemma E2B QAT weights (served by an unmodified llama-server on the GPU; faster-whisper and MMS-TTS on CPU — the same ears/mouth placement we chose, reached there by necessity since PyPI CUDA wheels cannot see the Tegra iGPU). End of speech → first reply audio, live-paced 16 kHz socket input, warm medians:

pipeline	warm TTFB	spread
HF speech-to-speech, defaults	~1.6 s	0.7–1.7 s
HF speech-to-speech, tuned (sentence batch 1)	~0.9 s	0.7–1.7 s
ours (Section 5.4)	0.65 s	reproducible legs

Table 7: HuggingFace speech-to-speech vs our pipeline on the same board and weights

The remaining difference is precisely the paper’s contribution list run in reverse: their chain is serial after end of speech — a full ASR pass, a full prompt prefill, the first sentence’s decode, and a whole-first-sentence VITS synthesis (their default even batches **THREE** sentences before the first TTS call; changing that one number is the 1.6 $\rightarrow$ 0.9 difference). Their first-audio latency therefore scales with reply length, where ours is $\sim O(1)$ after Section 4.2–4.4. VAD hang time is not the story on either side (their silence threshold is 64 ms).

Two qualifications, both in their favor and against. Their flagship TTS backends (Qwen3-TTS via CUDA-graph codec decoding, Kyutai’s Pocket) do stream natively — the whole-sentence behavior we measured is their CPU-viable tier — but the streaming tier is a different model architecture wanting a GPU, not a retrofit onto stock voices like Section 4.4. And their measured 0.9 s carried an artificially light ASR leg: the VAD only captured ~ 0.9 s fragments of our synthetic-voice question. On long input the comparison collapses entirely: a 31.8 s natural spoken question is segmented at breath pauses and answered INTO the user’s ongoing speech (first audio 25–28 s before end of speech), and even with perfect segmentation the post-speech chain scales with utterance length (their ASR leg alone measures 3.9 s for 31.8 s of audio on this CPU) — where prefill-under-speech holds ours at 0.65 s regardless (Section 4.2’s 929-token case is the measured proof).



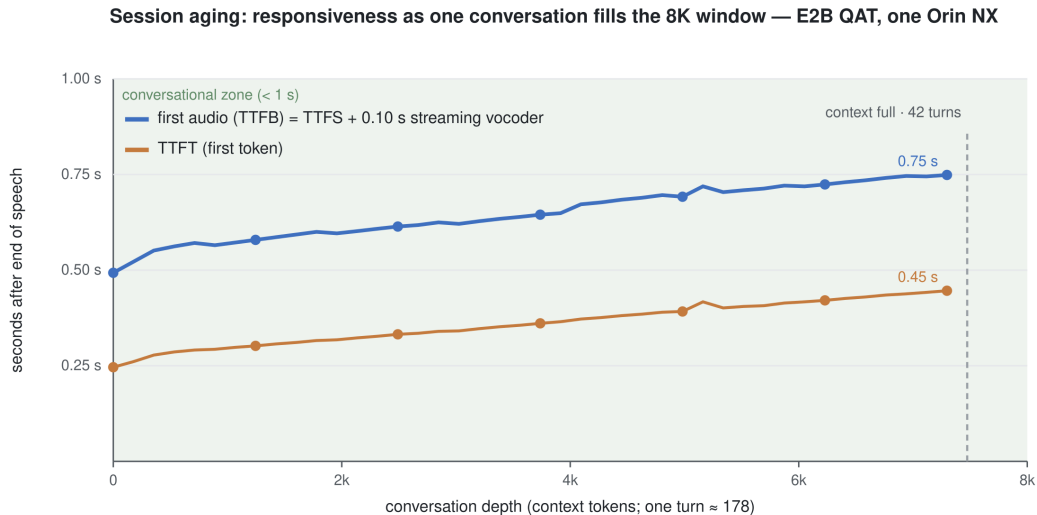
Same board, same E2B weights. HF floor = measured base-int8 ASR + 0.45 s downstream — its BEST case; real long input replies into ongoing speech.

Figure 5: The same board and E2B weights, swept over prompt length. Our TTFB is flat (0.65 s streamed-text; ≈ 1.0 s live-mic once the streaming-ASR final commit is included, still inside the conversational band). The HuggingFace perfect-VAD floor rises with the serial ASR pass. Points are measured; the HF line is its best case — real long input replies into ongoing speech, which is worse than the floor shown.

Figure 5 makes the divergence explicit. We swept six spoken questions of increasing length (3.8–32 s, synthesized and measured on the same board) and clocked each leg. Our prefill term is flat — and, at these lengths, nearly free in either placement: streamed tftt stays at 0.09–0.12 s across the range, and even a deferred one-shot prefill of the 90-word question costs 0.20 s. Natural speech arrives at ~ 3 tokens/s against a prefill rate of ~ 800 (E2B); anything a user can say in a turn is cheap to prefill. What the open turn hides at conversational lengths is therefore the ASR pass, not the prefill — prefill hiding earns its keep at dictation lengths and on the slower 12B tier (Section 4.2). The HuggingFace floor is not a line we could measure end-to-end — its VAD replies into our synthetic speech past ~ 15 s — so we compose its best case: the measured faster-whisper base-int8 ASR pass (1.20 s at 3.8 s of audio, rising to 3.52 s at 32 s) plus a 0.45 s downstream constant (prefill, first-sentence decode, first MMS-TTS call) anchored to their measured 0.9 s tuned point. Even granting perfect segmentation, the floor rises from ~ 1.6 s at a 5 s prompt to ~ 3.9 s at 30 s. The gap the reader sees is the ASR placement: run serially after end of speech it scales with the utterance; committed during speech it costs one final pass, a constant. Everything else — the LLM prefill, the first speakable unit’s decode, the vocoder — is constant in prompt length on both sides; the size of those constants (clause-first flushing, the streaming vocoder) is Section 4.3–4.4’s fight, already settled in Figure 3.

5.10 Session aging: does it hold as the context fills?

A fair question the single-turn numbers do not answer: does responsiveness decay over a long conversation as the 8K window fills? We measured it directly — one persistent E2B QAT session on the Orin (plain serve, no MTP, pinned clocks, SoC otherwise idle), driven turn after turn with fixed ~ 178 -token turns until the server reported the context full, which happened after 42 turns ($\sim 7,475$ tokens).



Plain serve, pinned clocks. TTFT 0.25 $\rightarrow$ 0.45 s and first audio 0.49 $\rightarrow$ 0.75 s across a full session — both stay inside the conversational band. Decode 39.6 $\rightarrow$ 28.6 tok/s.

Figure 6: TTFT and first audio (measured TTFS + the streaming vocoder’s 0.10 s first-PCM constant, Section 4.4) as one conversation fills the window. Both rise gently and slightly sub-linearly — TTFT 0.25 $\rightarrow$ 0.45 s, first audio 0.49 $\rightarrow$ 0.75 s — and both stay inside the conversational band for the entire session. Plain decode slows 39.6 $\rightarrow$ 28.6 tok/s (attention over the growing KV), still far above the ~ 3 tok/s real-time-listening floor (Section 6).

The growth is bounded because most layers use sliding-window attention — their cost is window-bounded and constant — so only the global layers pay for the accumulating KV. Absolute values reflect this fixed short-turn stimulus, chosen to fill the context at a constant rate; the load-bearing result is the slope: a full session adds only ~ 0.2 s to TTFT and ~ 0.26 s to first audio, and the pipeline still answers in under 0.75 s at context-full. The design point holds across the whole session, not just the first turn.

6 Discussion

What the cascade buys for its ~ 0.5 s of structural disadvantage: the brain is a commodity part. The same pipeline ran three Gemma 4 variants (E2B, E4B, 12B) without retraining anything; it processes a camera feed with the same open-turn mechanics that stream dictation (Section 4.2); it speaks Gemma’s full tool-calling protocol because the server passes control tokens through; and its answer quality is that of a 12B instruction-tuned text model, not what survived speech-topology training. Fluency, meanwhile, turned out not to require a new species of model — just refusing to waste the time the user’s own speech provides, and letting the model place its own pauses.

The A5000 study reframes the edge question. The usual framing is "how much worse is edge?"; measured, the answer in this regime is $1.7\times$ on latency, at $1/6$ th the power class — because the floor is set by clause decoding and vocoding, not by FLOPs. TTFB pursuit and throughput pursuit diverge here: past the point where prefill hides under speech and decode outruns speech playback (~ 3 tok/s suffices for real-time listening; even the 12B’s 8.3 tok/s clears it), more tokens/s stops buying experienced fluency. That is why this work no longer competes with llama.cpp on llama.cpp’s axis — and why a 6,000-line runner can hold the design point at all.

7 Limitations

- The dictation TTFB excludes the ASR final-commit pass (Section 5.4 gives the live-mic number, ~ 1.0 s to turn close); Moshi’s number includes its hearing. The persistent-whisper upgrade path narrows this and is measured future work (Section 8).
- The Moshi comparison is anchor-mismatched in Moshi’s favor and ours at once: its probe clock starts at the start of input streaming (fed silence, the full-duplex model initiates speech itself), ours at the end of user speech. Full-duplex design bounds Moshi’s turn response by roughly the same loop latency, so the citation is fair in spirit, but the rigorous head-to-head — stream a recorded question, anchor at its final word — remains future work. (The handshake question is resolved: excluded by construction.)
- Style pressure makes the 2B confidently wrong where it was vaguely right (Section 4.3); the clause-splitting prompt trades a quality edge for fluency at the smallest tier. Finetuning may recover both.
- All benchmarks use greedy decoding (the byte-identity discipline requires it). Sampling exists and composes with MTP distribution-exactly — the verify samples the target’s distribution and accepts a draft only on agreement, so speculation never alters the output distribution — but acceptance drops ($76.6\% \rightarrow 28\text{--}46\%$ at temperature 1.0 on the 2B) and sampled-mode latency/quality is not separately characterized here.
- Single-user, single-stream serving; no batching story.
- 8 GB verdict extrapolated from the NX 16GB (Section 5.6).
- Gemma 4’s native audio path is not usable as the pipeline’s ears, which is what makes the whisper lane load-bearing rather than a fallback. The capability boundary we measured (greedy decoding; Orin + A5000):

Configuration	Native-audio result	Status
E2B/E4B, speech	Original release: confabulates — output tracks the prompt, not the audio (all four independent pipelines failed on the same clip, each differently). QAT release: fixed — verbatim ASR, isolated to the repaired audio-encoder export plus its fake-quant activation ranges (Section 5.5); verified on E4B and E2B QAT (the E2B verbatim through the chunked streaming path; a single long span truncates on the smaller model).	
12B, speech only	Accurate ASR (JFK transcribed verbatim).	Usable for speech only.
12B, non-speech only	Confabulates or denies; describes the instrument named in the prompt.	Music/ambient require an external tagger.
12B + vision	Hearing disabled for all tested orderings; vision remains functional.	Use Whisper whenever vision is enabled.

Table 8: Native audio evaluation results.

8 Future Work

Finetune for human pause placement (shorter than grammatical clauses); persistent whisper service (flips the streaming-vs-endpass verdict unconditionally); integrate the streaming vocoder (Section 4.4) into the live-mic loop and re-measure the composed 0.65 s as one system; live-mic native ears (Section 5.5) — mic chunks as media frames aligned to a far-field front end’s speech segments, and the chunk dial validated across speakers and boundary placements; E2B QAT native-audio verification; the rigorous Moshi head-to-head (stream a recorded question, anchor the clock at its final word — Section 7); re-run the A5000 cross-silicon table with streaming TTS on both sides; Nano 8GB SKU validation; barge-in latency characterization; camera+voice concurrent sessions (the A/V exclusion policy exists, the latency study does not).

9 Conclusion

Fluent and cohesive do not require choosing. A 20 W board running unmodified open-weight models answers 0.65 s after you stop talking — because the pipeline stops wasting the seconds the user’s own speech offers (prefill under speech), lets the model create the places speech can begin (the LLM as clause splitter), and measures the thing the

ear actually judges (TTFB, not TTFT). The gap to purpose-trained speech-to-speech models is real, structural, and — at 2–4× against a species that cannot swap its brain, read a camera, or call a tool — a good trade for most embodied products.

A Reproducibility

Every number traces to a committed harness: the repository’s bench/ directory carries the dictation clients (client-side clocks, three delivery modes), the standard 929-token stimulus, the TTS timing scripts, the memory samplers (anon+nvmap), the serve-prefill bench harnesses for both machines, and the clause-flushing glue that makes the pipeline a single shell command; docs/voice-pipeline.md is the runnable reproduction guide (exact command lines, protocol rules, and the honest gaps). The streaming vocoder ships in our piper fork (split tool + streaming API + tests). The engineering journals include the falsified attempts. Replies are byte-identical across delivery modes and quantization levels by gate.

References

- [1] S. Sharma, “Voice AI Latency: What’s Fast, What’s Slow, and How to Fix It,” Hamming AI, Jan. 12, 2026. [Online]. Available: <https://hamming.ai/resources/voice-ai-latency-whats-fast-whats-slow-how-to-fix-it>. Archived at: <https://web.archive.org/web/20260511153827/https://hamming.ai/resources/voice-ai-latency-whats-fast-whats-slow-how-to-fix-it>
- [2] Hamming AI, “Voice Agent Evaluation Metrics: Definitions, Formulas & Benchmarks,” Jan. 18, 2026. [Online]. Available: <https://hamming.ai/resources/voice-agent-evaluation-metrics-guide>. Archived at: <https://web.archive.org/web/20260707200723/https://hamming.ai/resources/voice-agent-evaluation-metrics-guide>
- [3] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, N. Zeghidour, “Moshi: A speech-text foundation model for real-time dialogue,” *arXiv preprint arXiv:2410.00037*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.00037>
- [4] T. Stivers, N. J. Enfield, P. Brown, C. Englert, M. Hayashi, T. Heinemann, G. Hoymann, F. Rossano, J. P. de Ruiter, K.-E. Yoon, and S. C. Levinson, “Universals and cultural variation in turn-taking in conversation,” *Proceedings of the National Academy of Sciences*, vol. 106, no. 26, pp. 10587–10592, 2009. doi:10.1073/pnas.0903616106.
- [5] Kyutai Labs, “Moshi,” GitHub repository. [Online]. Available: <https://github.com/kyutai-labs/moshi>
- [6] NVIDIA, “NVIDIA L4 Tensor Core GPU,” Datasheet. [Online]. Available: <https://www.nvidia.com/en-us/data-center/l4/>
- [7] Z. Xie and C. Wu, “Mini-Omni: Language models can hear, talk while thinking in streaming,” *arXiv preprint arXiv:2408.16725*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.16725>
- [8] Z. Xie and C. Wu, “Mini-Omni2: Towards open-source GPT-4o with vision, speech and duplex capabilities,” *arXiv preprint arXiv:2410.11190*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.11190>
- [9] Pipecat (Daily), GitHub repository. [Online]. Available: <https://github.com/pipecat-ai/pipecat>
- [10] LiveKit, “Agents,” GitHub repository. [Online]. Available: <https://github.com/livekit/agents>
- [11] Hugging Face, “speech-to-speech (v0.2.10),” GitHub repository. [Online]. Available: <https://github.com/huggingface/speech-to-speech>
- [12] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” GitHub repository. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [13] D. Macháček, R. Dabre, and O. Bojar, “Turning Whisper into real-time transcription system,” *arXiv preprint arXiv:2307.14743*, 2023. Presented at IJCNLP-AACL 2023 System Demonstrations. [Online]. Available: <https://arxiv.org/abs/2307.14743>
- [14] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: Speculative sampling requires rethinking feature uncertainty,” *arXiv preprint arXiv:2401.15077*, 2024. Presented at ICML 2024. [Online]. Available: <https://arxiv.org/abs/2401.15077>
- [15] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” *arXiv preprint arXiv:2401.10774*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.10774>

-
- [16] Arrow Electronics, “Jetson Orin NX 16GB module pricing,” quotation obtained Jul. 6, 2026. [Online]. Available: <https://www.arrow.com>
 - [17] Codesota, “TTS models, split by track, ranked by preference.” [Online]. Available: <https://www.codesota.com/guides/tts-models>
 - [18] Cortexist, “little-gemma,” GitHub repository. [Online]. Available: <https://github.com/cortexist/little-gemma>